CITY
ST GEORGE'S
UNIVERSITY OF LONDON

School of Science & Technology

IN1011 Operating Systems
Stage 1 Mock Examination B (Solutions)

May/June

Examination duration: **90** minutes

Division of marks: Answer **3** of the **4** questions (20 marks for each question). If more than **3** questions are answered, the best **3** will be counted. Total mark is out of **60**

Other instructions: All necessary working must be shown.

BEGIN EACH QUESTION ON A FRESH PAGE

Number of answer books to be provided: 1
Calculators permitted: Yes
Dictionaries permitted: None
Can question paper be removed from the examination room: No

Additional materials: **None**

Question 1

a. Describe 5 distinct scenarios that each illustrate how the OS relies on interrupts to perform its activities. Each scenario should clearly state how the interrupt arises (i.e. the situation/event that makes the interrupt necessary), the hardware that triggers the interrupt signal, and what the OS is expected to do in response to the interrupt.

[15 Marks]

Solution: There are several possible solutions. For example, any 5 of the following answers, or analogous answers, will be accepted:

i) moving your mouse to move a cursor on the screen. When the mouse is moved **[1/3]** an interrupt signal is generated by the I/O module for the mouse**[1/3]**. The OS interrupt handler for mouse movements will use position data from the mouse to position the cursor when it is rendered on-screen**[1/3]**;

ii) typing a document in MS Word. Keyboard presses/strokes generate interrupt signals **[1/3]** by the I/O module connected to the keyboard **[1/3]**. The OS interrupt handler for the keyboard will notify the MS Word application about which keys are pressed, so that the corresponding alphanumeric characters are entered into the document and rendered on-screen**[1/3]**;

iii) playing a videogame using a game controller. Pressing buttons on the controller and moving analog sticks with your thumbs generate interrupt signals **[1/3]** by the I/O module for the controller **[1/3]**. The OS interrupt handler for the controller will notify the game about which buttons have been pressed as well as the positions of the analog sticks, so that your controller actions are registered in-game **[1/3]**;

iv) streaming a Netflix show. When a network connection is made to Netflix's servers and data is arriving at your computer from Netflix, the arrival of data generates interrupt signals **[1/3]** by the I/O module associated with your network card **[1/3]**. The OS interrupt handler for network connections/data will notify the web browser application and forward the streaming data that has arrived, which will be processed and played as video data **[1/3]**;

v) writing with a digital pen and tablet. The movement of the pen across the tablet generates interrupt signals **[1/3]** by the I/O module for the pen+tablet**[1/3]**. The OS interrupt handler for the pen+tablet notifies the drawing application being used, and forwards data about the position of the pen and the pressure being applied to the pen. The application can use this to render the pen strokes appropriately**[1/3]**;

vi) printing a document. When a printer finishes a print job, an interrupt signal is generated **[1/3]** by the I/O module for the printer**[1/3]**. The OS interrupt handler for the printer will notify the application that initiated the print job, and forward any status data related to how successful the print job was **[1/3]**;

vii) receiving a WhatsApp message. When network data arrives for the WhatsApp application **[1/3]** the I/O module associated with the network card will generate an

interrupt**[1/3]**. The OS interrupt handler will identify the data as being for WhatsApp and will forward the data for WhatsApp to process and display the message**[1/3]**.

viii) making a Zoom call using a mic and webcam. When the mic and camera are enabled**[1/3]**, they continuously generate interrupts by I/O modules connected to these devices**[1/3]**. The OS interrupt handlers for the mic and webcam notify Zoom and forward audio-visual data from these devices, which Zoom then uses and packages into data that will be sent to other Zoom clients on the call**[1/3]**.

b. Let $\boldsymbol{x}$ be a positive integer (e.g. 1, 2, 3, etc.). Furthermore, let $\boldsymbol{x1}$ and $\boldsymbol{1x}$ be 2-digit numbers in base $\boldsymbol{x}$+1. What is $\boldsymbol{x1} + \boldsymbol{1x}$ in base $\boldsymbol{x}$+1? Show how you arrived at your answer using the definition of what it means to write numbers in base $\boldsymbol{x}$+1.
(*hint: write* $\boldsymbol{x1}$ *in powers of* $\boldsymbol{x}$*+1, write* $\boldsymbol{1x}$ *in powers of* $\boldsymbol{x}$*+1, then add these together*)
[2 Marks]

Solution: The answer is **110**. To see this, note that $\boldsymbol{x1}$ means $\boldsymbol{x}(\boldsymbol{x}+1)^1 + \boldsymbol{1}(\boldsymbol{x}+1)^0$ and $\boldsymbol{1x}$ means $\boldsymbol{1}(\boldsymbol{x}+1)^1 + \boldsymbol{x}(\boldsymbol{x}+1)^0$ **[1/2]**. So, adding these together and grouping/factorising the answer in powers of $(\boldsymbol{x}+1)$ gives

$$\boldsymbol{1}(\boldsymbol{x}+1)^2 + \boldsymbol{1}(\boldsymbol{x}+1)^1 + \boldsymbol{0}(\boldsymbol{x}+1)^0.$$

Reading off the coefficients, the answer is **110** **[1/2]**.

c. The command "**`chmod 522 mybinaryfile`**" is executed in a Bash terminal.

i. In what base is the 3-digit number **`522`** written in? **[1 Mark]**

Solution: Each digit can take on values from 0 to 7, so this is base 8 (i.e. Octal)**[1/1]**.

ii. How many distinct permission settings can be carried out using such 3-digit numbers with **`chmod`**? How did you deduce your answer?
[2 Marks]

Solution: Each digit can take on 8 possible values**[1/2]**. So, the number of possible 3-digit numbers here is $8 \times 8 \times 8 = 512$**[1/2]**.

Question 2

a. Consider what happens when the following program G() is executed.

```
G( )
{
    int c = 0;
    int id = fork();

    if( id==0 ){ c = c+1; }
    else{
        id=fork();
        c=c+1;

        if( id==0 ){ c = c+1; }
     }
}
```

i. How many processes are there in total? **[1 Mark]**

Solution: fork() is called twice by the first process to be created by the OS, so there are 3 processes in total**[1/1]**.

ii. How many parent processes? **[1 Mark]**

Solution: fork() is called twice by the same process, so there is only 1 parent process**[1/1]**.

iii. How many child processes? **[1 Mark]**

Solution: there are 2 children, each one created from a call to fork()**[1/1]**.

iv. What are the different final values of c? **[3 Marks]**

Solution: The three processes each have a copy of c that is initialised to 0. The parent process increments their copy of c only once, to a value of 1**[1/3]**. The first child process increases their copy of c only once, also to a value of 1**[1/3]**. The second child updates their copy of c twice, to a value of 2**[1/3]**.

v. What final value of c is associated with the final process created? **[2 Marks]**

Solution: the second child is the final process created**[1/2]**, so their c has a final value 2 (see previous answer to iv)**[1/2]**.

vi. What final value of c is associated with the parent of the process in the previous question v? **[2 Marks]**

Solution: the parent process **[1/2]** has a value of 1 for their c**[1/2]**.

vii. Why must G( ) cause interrupts during its execution? **[1 Mark]**

Solution: fork() is an API method that only the OS kernel can execute. So, each time G() calls fork() it is making a system call – i.e. a software interrupt**[1/1]**.

b. What is *the memory hierarchy*? Your answer should clearly indicate 3 ways the hierarchy constrains modern computers. **[4 Marks]**

Solution: The memory hierarchy is an organisation of memory modules in modern computers, based on trade-offs between performance, cost, and memory size of these modules**[1/4]**. Faster modules (e.g. cache, registers) are typically significantly more expensive (per byte) **[1/4]** than slower modules (e.g. RAM, harddisk) **[1/4]**. So, significantly smaller sizes (in bytes) of faster modules are used in computers, compared with significantly greater sizes of slower modules**[1/4]**.

c. What is *the principle of spatial locality*? **[2 Marks]**

The principle of spatial locality is the phenomenon that when an instruction is fetched from memory and executed, nearby instructions**[1/2]** in memory are likely to also be executed soon afterwards**[1/2]**.

d. Using an example involving computer hardware resources, illustrate how *the principle of spatial locality* can be used to partly overcome *memory hierarchy* constraints. **[3 Marks]**

Solution: the CPU typically spends time waiting for instructions to be fetched from RAM to be executed. In particular, there will be a lot of time the CPU spends idle if it has to wait for every instruction to be fetched from RAM**[1/3]**. However if, instead, all of the instructions were loaded into, and fetched from, faster memory (such as registers), the CPU will be less idle but the computer would be prohibitively expensive because of the amount of "register" memory needed to do this**[1/3]**. The compromise is for the computer architecture to rely on the *principle of spatial locality*. So, when an instruction is fetched from RAM, nearby instructions are buffered**[1/3]** into smaller faster memory, just in case these instructions also need to be executed soon afterwards.

Question 3

a) What is the *Effective Access Time* (EAT) in a demand-paged system? Explain.

[4 Marks]

Solution: Effective Access Time (EAT) is the average time taken to service a memory reference. If there were never any page-faults (p = 0) this would just be the base hardware memory access time (actual RAM access time), but when a page-fault happens, the following extra time is spent: time taken to execute page-fault trap + time taken to swap pages in and out from disk + time taken to return from trap and restart the process which made the memory reference. We call this combined extra time the page-fault service time

b) Consider a system in which 100% of page-faults require a page to be swapped in from disk and 50% of page-faults require a victim page to be swapped out. If the page size is 4KiB and disk read and write speeds are both 300 MiB per second, calculate the *average page-fault service time* (assume all overheads apart from swap time are negligible). Give your answer to two significant figures.

[6 Marks]

Solution: the average page fault time is the average time that it will take. As 100% of page faults require a swap in, that will always be necessary, but only 50% of the pages require a swap out, so 50% of that time will be added:

= 1 * 4 [kB] /300 [MBs] + 0.5 * 4 [kB] /300 [MBs]

= 13.33 [ms] + 6.66 [ms] = 19.9 [ms] ~ 20 [ms]

c) For a system with page size 4,096 bytes and a process with 123,456 bytes, calculate:

i) number of pages used, **[2 Marks]**

ii) internal fragmentation. **[2 Marks]**

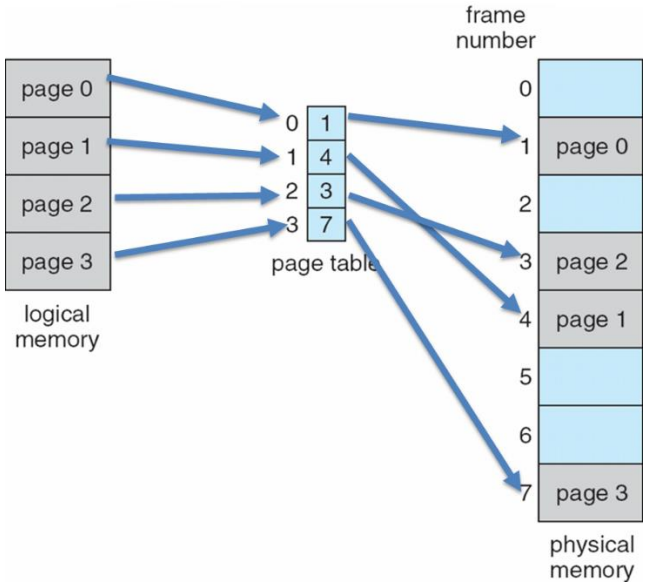
Solution: 123,456 / 4096 = 30.14 -> 31 pages (30 pages + 576 bytes)

Internal fragmentation = 4096 – 576 = 3,520 bytes or 0.86 page

d) In a virtual memory system based on paging, what is a page table? Draw a diagram to show how a page table (without any translation look-aside buffer) is used to translate a virtual memory address to a physical memory address.

[6 Marks]

Solution: A page table is a data structure used by a virtual memory system in a computer to store mappings between virtual addresses and physical addresses. Virtual addresses are used by the program executed by the accessing process, while physical addresses are used by the hardware, or more specifically, by the random-access memory



Question 4

a) A process that requires 8.2 MB of space is swapped to a hard disk. The transfer rate to the disk is 108.4 MB/s and there is a further latency of 7.3 microseconds. Calculate the total Context Switch Swapping time (that is swap out and swap in) of this process. Write down your answer in milliseconds.

[5 Marks]

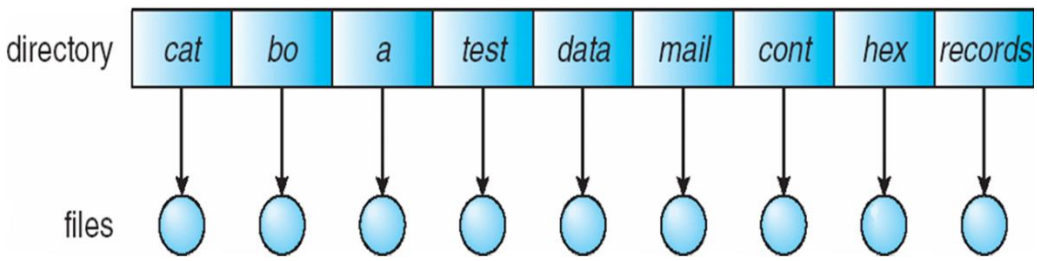
Solution: 8.2 [MB] / 108.4 [MB/s] = 75.645 [ms]

75.645 [ms] x 2 + 0.072 [ms] x2 = 151.43 [ms]

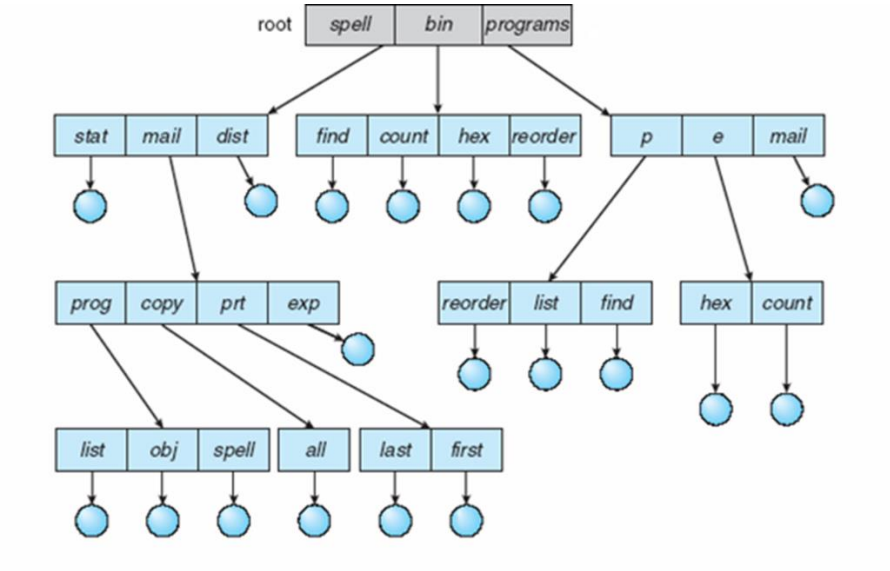
b) Describe the way files are organised in a single-level directory. What are the main two problems of this structure? Describe another directory structure that addresses the problems of the single-level directory. Use diagrams to support your answer.

[7 Marks]

Solution: A single level directory is the simplest and most basic way to organise files. The file space consists of a single folder (not the disk directory) and inside all the files are named, one single main monolithic structure with many file names, The most obvious shortcomings are that there can only be a single name for a file, once a file has been saved (like TEMP or TEST) it is not possible to save another with the same name. Then, it is not possible to organise files in groups or group them in any way. This is a very limited option..



A more effective way to save files is in a tree structure, in which folders can contain files as well as other folders. In this way it is possible to organise a large number of files, which can have the same name as long as they are located in different folders.



c) A group of files that exists in a directory have the permissions shown in the image below. Identify the permissions in a numeric way.

```
-rw--wx--x
--w-r-xr--
--------rwx
```

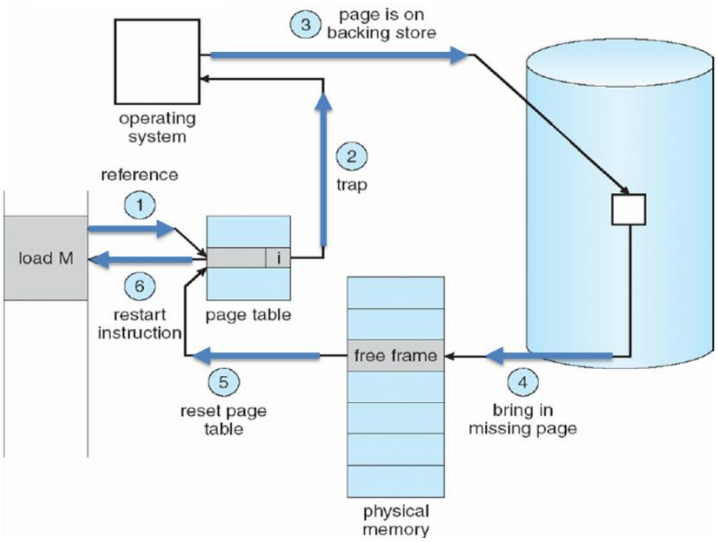
[3 Marks]

Solution: 6 3 1, 2 5 4, 0 0 7

d) In a demand-paged system, what is a *page-fault*? Outline the procedure by which a page-fault is handled. You can use diagrams to support your answer.

[5 Marks]

Solution: A method of virtual memory management can bring a page into memory ("page in") only when it is needed. When the page is needed the process tries to access an address in that page. If the page is not in memory it is considered "page-fault" and then the page has to be brought to memory from memory. This is considered as a "lazy swapper" – never swaps a page into memory unless the page is used (demanded). The steps followed in a page fault follow the diagram below:


page is on backing store
operating system
reference
trap
load M
i
page table
restart instruction
free frame
reset page table
bring in missing page
physical memory

End of Exam